

lab6

要求：在原先os-part4仓库中完成内存相关调用 `test_vm_fifo` `test_vm_lru`

▼ 前置操作

- 运行这个指令，切换到part6的分支上，并在这个分支上创建一个新的分支

```
git checkout part6-page-replace
```

- 看助教的参考步骤，我们知道在前两步，两个测试点要做的事情都一样，所以我们不必现在就进行分支，在这个part6的主分支上，将步骤一和步骤二完成。

▼ 步骤一：

- 实现 `set_max_page_in_mem`、`get_page_swap_count` 两个系统调用：在进程管理结构里添加相应字段，其中 `max_page_in_mem` 用于限制单个进程最多能使用的mmap映射区域的页面数，而 `page_swap_count` 用于记录进程在mmap映射区域发生的换出行为次数。（具体逻辑可以看测试样例的注释说明）

▼ set_max_page_in_mem：

- 这是一个系统调用，因此我们仍需要再 `syscall.c` 中对齐进行注册，接下来，我们要在 `sysproc.c` 中完成它的定义。
- 这个调用用于限制单个进程最多可以使用的 `mmap` 映射区域，超过了这个数量时，则触发页面调换。既然涉及到了进程的内容，我们去 `proc.h` 中，查看一下有关进程和 `mmap` 的定义情况。

```
#define MAX_VMA 16 // 每个进程最多16个映射区域

struct VMA {
    uint64 vm_start;
    uint64 vm_end;
    int prot;
    int flags;
    uint64 vm_off;
    struct VMA *vm_next, *vm_prev;
};
```

```

. . .

// Per-process state
struct proc {
    struct spinlock lock;

    // p->lock must be held when using these:
    enum procstate state;           // Process state
    struct proc *parent;           // Parent process
    void *chan;                     // If non-zero,
    sleeping on chan                // If non-zero,
    int killed;                     // If non-zero,
    have been killed                // Exit status to
    int xstate;                     // be returned to parent's wait
    int pid;                        // Process ID

    // these are private to the process, so p->lock
    // need not be held.
    uint64 kstack;                 // Virtual address
    of kernel stack               // Size of process
    uint64 sz;                     // memory (bytes)
    pagetable_t pagetable;         // User page table
    pagetable_t kpagetable;        // Kernel page table
    struct trapframe *trapframe;   // data page for
    trampoline.S                  // swtch() here
    struct context context;         // to run process
    struct file *ofile[NOFILE];    // Open files
    struct dirent *cwd;            // Current directory
    char name[16];                 // Process name
    (debugging)                    // trace mask
    int tmask;

```

```

    struct VMA head;
};

```

这里我们发现，在最开始，定义的vma最大数量是一个宏，这个我们没法修改，但事实上我们这里需要实现的是一个新的变量，它用于记录当前内存中最多可以使用多少个vma，所以接下来我们就必须给 `struct proc` 自己添加一个额外的变量 `int max_page_in_mem`，同时要给出一个记录目前有多少个vma在内存中的变量 `int cur_page_in_mem`，并且需要我们自己追踪在进程全过程中的它的周期。

进入 `proc.c`，我们寻找这个变量的全周期。（不过如果我们直接阅读测试样例，你就发现其实做这个变量的管理也没那么关键，因为不涉及 `fork` 等操作，所以接下来的很多操作只是出于健壮考虑）

首先应该是进程创建时，对这个变量进行初始化。

```

static struct proc *
allocproc(void){
    ...
    p->kstack = VKSTACK;
    p->max_page_in_mem = MAX_VMA;
    p->cur_page_in_mem = 0;
    p->head.vm_prev = &p->head;
    p->head.vm_next = &p->head;
    ...
}

```

其次是在回收进程时将其清空。

```

static void
freeproc(struct proc *p)
{
    ...
    p->pid = 0;
    p->parent = 0;
    p->max_page_in_mem = 0;
    p->cur_page_in_mem = 0;
    p->name[0] = 0;
}

```

```
    ...  
}
```

然后是在fork时的复制。

```
// Create a new process, copying the parent.  
// Sets up child kernel stack to return as if f  
rom fork() system call.  
int fork(void)  
{  
    ...  
  
    np->parent = p;  
  
    // copy tracing mask from parent.  
    np->tmask = p->tmask;  
  
    np->max_page_in_mem = p->max_page_in_mem;  
    np->cur_page_in_mem = p->cur_page_in_mem;  
  
    ...  
}
```

随后我们到测试样例中去看一下这个函数在用户态是怎么被定义的，这能告诉我们应该如何传递参数。

```
set_max_page_in_mem(4);
```

好，接下来我们只需要在 `sysproc.c` 中添加这个简单的系统调用即可。

```
uint64  
sys_set_max_page_in_mem()  
{  
    int n;  
    if (argint(0, &n) < 0)  
    {  
        return -1;  
    }  
}
```

```

myproc()->max_page_in_mem = n;
return 0;
}

```

▼ get_page_swap_count:

- 同样地，我们需要在进程的全周期添加一个新的变量 `int page_swap_count` 用于记录该进程中发生的所有的页面换出行为。记得在 `syscall.c` 中添加这个 `sys_get_page_swap_count` 的注册内容。
- 但是你要记得，在fork时，不应该将父进程的页面换出数量加进来，所以这块这么写：`np->page_swap_count = 0;`

```

uint64
sys_get_swap_count()
{
    return myproc()->page_swap_count;
}

```

▼ 步骤二:

- 由于真实交换区可能位于磁盘上，涉及到文件操作，较为麻烦，因此本实验中可以使用**内存里模拟的交换区**进行替代。实现内存里模拟的交换区，用于存放**各个进程mmap映射区域换出的页面**，在进程需要访问交换区里的页面时，进行**换入操作**。
 1. 全局交换区可参考数据结构设计：由全局数组/链表结构组织的**slot**，每个slot存放一个 `PAGESZ` 大小的数据，并维护当前slot的状态（已使用/未使用）、数据的所有者PID、数据对应的起始地址addr等。
 - a. `alloc_global_swap_slot`：从全局交换区里分配一个空闲的slot（用于之后换出页面内容）
 - b. `free_global_swap_slot`：将使用完毕的slot归还至全局交换区（换入内容后释放资源）
 2. 对进程 `mmap` 区域的每个页面，维护**标志位**表明页面**是否处于交换区**（若处于交换区，则认为是内存不够发生了换出），可以考虑修改 `struct VMA` 结构体以维护**mmap区域对应的各个Page的信息**：Page的起始地址，Page当前的状态（在交换区/在内存/未使用），Page的末次访问时间（用于LRU）和本次进入内存时间（用于FIFO），etc.；

- 参考这个要求，一个交换区是全局变量，这个交换区的每个结构体应该具有来源 `proc` 的 `PID` 等信息，因为其作为一个物理页面的模拟，它还需要能够完整记录一整个页面的数据，为了方便后续的管理，由于我们将真实交换区仍然模拟为虚拟内存中的结构，所以我们直接将其定义在 `vm.h` 下，方便这些虚存管理函数对齐操作。

这里的`alloc`与`free`我们可以参考 `allocproc` 和 `freeproc`，分配时返回一个结构体或全局交换区中的编号，释放时什么也不用返回。

```
void mock_swap_init(void);
int alloc_global_swap_slot(int pid, uint64 vaddr);
// 从全局交换区里分配一个空闲的slot（用于之后换出页面内容）
void free_global_swap_slot(int idx); // 将使用完毕
的slot归还至全局交换区（换入内容后释放资源）
struct swap_slot
{
    int used;           // 是否有效
    int pid;            // 进程ID
    uint64 vaddr;       // 该页面原本对应的虚拟地址
    char data[PGSIZE]; // 该页面数据的实际缓冲区
};
#define MAX_SWAP_SLOTS 10 // 由于测试样例只有8个页面
需要来回交换
```

因为我们的交换区是一个全局变量，可能涉及多个进程共享同一个区的现象，如果我们不能保证操作的原子性，可能出现两个进程分配到同一个 `slot` 槽位等竞态问题。所以在全局交换区变量的设计上，我们加一个锁吧。（如果我们只是为了通过测试点，这个锁应该也不用写，我看测试样例没有fork的问题）

```
struct
{
    struct spinlock lock;
    struct swap_slot slot[MAX_SWAP_SLOTS]; // 因为测试
样例只有8个页面，所以全局交换区设置为10个slot以防止越界
} mock_swap;

void mock_swap_init(void)
{
```

```

initlock(&mock_swap.lock, "mock_swap");
for (int i = 0; i < MAX_SWAP_SLOTS; i++)
{
    mock_swap.slot[i].used = 0;
    mock_swap.slot[i].pid = -1;
    mock_swap.slot[i].vaddr = 0;
    memset(mock_swap.slot[i].data, 0, PGSIZE);
}
}

```

接下来是助教给出的两个函数的定义，这两个函数作为该全局交换区的对外接口，管理这个交换区 `slot` 的分配与撤销。

```

int alloc_global_swap_slot(int pid, uint64 vaddr)
{
    acquire(&mock_swap.lock);
    for (int i = 0; i < MAX_SWAP_SLOTS; i++)
    {
        if (!mock_swap.slot[i].used)
        {
            mock_swap.slot[i].used = 1;
            mock_swap.slot[i].pid = pid;
            mock_swap.slot[i].vaddr = vaddr;
            release(&mock_swap.lock);
            return i;
        }
    }
    release(&mock_swap.lock);
    return -1; // 没有可用的slot
}

void free_global_swap_slot(int idx){
    acquire(&mock_swap.lock);
    if(idx >= 0 && idx < MAX_SWAP_SLOTS){
        mock_swap.slot[idx].used = 0;
        mock_swap.slot[idx].pid = -1;
        mock_swap.slot[idx].vaddr = 0;
        memset(mock_swap.slot[idx].data, 0, PGSIZE);
    }
}

```

```

    }
    release(&mock_swap.lock);
}

```

之后，像这种全局变量的初始化，我们把它放在main.c中即可。

```

void main(unsigned long hartid, unsigned long dtb_
pa)
{
    ...
    timerinit();    // init a lock for timer
    trapinithart(); // install kernel trap vector,
including interrupt handler
    procinit();
    mock_swap_init();
    plicinit();
    ...
}

```

- 之后，根据流程，我们需要在VMA中额外维护一个成员变量，用于展示这个 `mmap` 映射的页，在此之前，我们先看一下在这个仓库下的 `mmap` 是怎么被实现的。首先，`sys_mmap` 的作用只是解析参数，我们可以看到真正起作用的是 `mymmap`。

```

uint64 mymmap(int fd, uint64 addr, uint64 len, int
prot, int flags, uint64 offset)
{
    struct proc *p = myproc();
    struct VMA *vma;
    // mmap映射的区域从进程的堆顶向上找，并从最顶端的TRAMPOLINE|TRAPFRAME处向下
    uint64 start = p->sz, end = TRAPFRAME;
    if (start - end < len)
        addr = -1;
    // 找到目前该进程中mmap链表的最靠低地址处的头
    for (vma = p->head.vm_next; vma != &p->head; vma
= vma->vm_next)
    {

```



```

        if (end - vma->vm_end >= len)
            break;
        end = vma->vm_start;
    }
    // 看看我们希望分配的这个映射内存与目前堆顶到mmap链表底
    之间的空隙是否够大
    if (start - end < len)
        addr = -1;
    addr = end - len;
    // 这个调用实际上从全局的空闲vma池中分配出一个来接到当前
    进程的p->head下
    if ((vma = allocshare()) == 0)
        return -1;
    // 根据传入的参数初始化这个vma
    vma->vm_start = addr;
    vma->vm_end = vma->vm_start + len;
    vma->prot = prot;
    vma->flags = flags;
    vma->vm_off = offset;

    // 挂载到当前进程的head尾部前一块
    struct VMA *cnt;
    for (cnt = p->head.vm_next; cnt != &p->head; cnt
    = cnt->vm_next)
        if (cnt->vm_start < vma->vm_start)
            break;
    cnt->vm_prev->vm_next = vma;
    vma->vm_prev = cnt->vm_prev;
    vma->vm_next = cnt;
    cnt->vm_prev = vma;

    return addr;
}

```

可以看到这个调研完全没有考虑到任何分页相关的内容，它只是给vma设置好了开始结束地址，但我们在 `mmap` 之后，根据要求，需要对 `mmap` 的地方进行“页”级别的置换。也就是说我们应该在这个调用中，对我们的成员变量做维护。

先定义好这个新的结构。

```
struct VMA_page
{
    int status; // 0: not used, 1: in memory, 2: swapped out
    uint64 vaddr;
    int swap_slot_idx; // 如果status为2, 记录对应的全局交换区
    uint64 last_in_mem_time; // 该页面进入内存的时间戳, 用于FIFO算法
    uint64 last_access_time; // 该页面最后一次被访问的时间戳, 用于LRU算法
};

struct VMA
{
    uint64 vm_start;
    uint64 vm_end;
    int prot;
    int flags;
    uint64 vm_off;
    struct VMA_page pages[10]; // 该映射区域内的页面信息, 测试样例就8页
    struct VMA *vm_next, *vm_prev;
};
```

随后我们处理一下这个新成员变量的全周期, 只考虑通过测试点, 我们只在 `mmap` 发生时对其进行初始化操作。

```
uint64 mymmap(int fd, uint64 addr, uint64 len, int prot, int flags, uint64 offset)
{
    ...
    if ((vma = allocshare()) == 0)
        return -1;
    vma->vm_start = addr;
    vma->vm_end = vma->vm_start + len;
```

```

vma->prot = prot;
vma->flags = flags;
vma->vm_off = offset;

// 这块用于处理VMA_page的初始化
int npages = (len + PGSIZE - 1) / PGSIZE; // 向上取整计算需要的页面数
for (int i = 0; i < npages; i++)
{
    vma->pages[i].status = 0; // 初始状态为not used
    vma->pages[i].vaddr = vma->vm_start + i * PGSIZE;
    vma->pages[i].swap_slot_idx = -1; // 初始时没有对应的交换区slot
    vma->pages[i].last_in_mem_time = 0;
    vma->pages[i].last_access_time = 0;
}

...
}

```

▼ 步骤三：

- 接下来是实现swap。
 1. `swap_out`：将mmap映射区的页面内容拷贝到交换区的桶内，（并释放被拷贝的页面内存）
 2. `swap_in`：将交换区桶内的页面内容拷贝到进程mmap映射区指定页面上，（并预先分配内存）
- 这两个调用应该是处理缺页异常之后的操作，但应该放在 `vm.c` 中，因为我们的全局交换区也在这里定义，所以我们在 `vm.h` 中给出两个函数的定义，在 `trap.c` 中直接使用即可。

这两个函数可以参考 `uvmmalloc`，而对于 `swap_out`，由于 `vmunmap` 中已经将对应 `pte` 设为0，所以就不用我们再手动进行valid位清除。但不考虑页面的权限了，在测试样例中没有涉及太多考虑。

```

int swap_out(struct proc *p, struct VMA_page *page)

```

```

{
    int idx = alloc_global_swap_slot(p->pid, page->v
addr);
    if (idx < 0)
    {
        return -1;
    }
    // 找到该换出页的PTE，获取物理地址，复制到交换区
    pte_t *pte = walk(p->pagetable, page->vaddr, 0);
    if (pte == NULL || (*pte & PTE_V) == 0)
    {
        return -1;
    }
    uint64 pa = PTE2PA(*pte);
    memmove(mock_swap.slot[idx].data, (char *)pa, PG
SIZE);

    // 解除映射，更新PTE和页表项状态
    if (walk(p->kpagetable, page->vaddr, 0) != NULL)
    {
        vmunmap(p->kpagetable, page->vaddr, 1, 0); //
这一步时先不释放物理页，只是解映射内核页表
    }
    vmunmap(p->pagetable, page->vaddr, 1, 1);
    page->status = 2; // 标
记为交换出
    page->swap_slot_idx = idx; // 记
录交换区slot索引

    p->cur_page_in_mem--;
    p->page_swap_count++;
    return 0;
}

int swap_in(struct proc *p, struct VMA_page *page)
{
    if (page->swap_slot_idx < 0 || page->swap_slot_i
dx >= MAX_SWAP_SLOTS)

```

```

{
    return -1;
}
// 从交换区复制回物理内存，更新PTE和页表项状态
char *mem = kalloc();
if (mem == NULL)
{
    return -1;
}
memmove(mem, mock_swap.slot[page->swap_slot_idx].data, PGSIZE);
if (mappages(p->pagetable, page->vaddr, PGSIZE,
(uint64)mem, PTE_W | PTE_R | PTE_U) != 0)
{
    kfree(mem);
    return -1;
}
if (mappages(p->kpagetable, page->vaddr, PGSIZE,
(uint64)mem, PTE_W | PTE_R) != 0)
{
    vmunmap(p->pagetable, page->vaddr, 1, 1);
    kfree(mem);
    return -1;
}

int idx = page->swap_slot_idx;
free_global_swap_slot(idx); // 释放交换区slot
page->status = 1;           // 标记为在内存中
page->swap_slot_idx = -1;   // 清除交换区slot索引

p->cur_page_in_mem++;
return 0;
}

```

在 `vm.h` 中添加定义，并加上。

```

struct proc;
struct VMA_page;

```

- 之后，我们进行缺页异常相关的处理。这边的逻辑是这样：`mmap`了8个页面以后，第一次试图写入一定出现缺页异常，此时会进行下面的流程

```
void
usertrap(void)
{
    ...
    else if (r_scause() == 13 || r_scause() == 15) {
        uint64 va = r_stval();
        struct proc *p = myproc();
        va = PGROUNDDOWN(va);
        struct VMA *vma;
        // 在p的vma链表中寻找这一缺页地址的对应vma结构体
        for(vma = p->head.vm_next; vma != &p->head; vma = vma->vm_next)
            if(va >= vma->vm_start && va < vma->vm_end)
                break;
        if(vma == &p->head)
        {
            p->killed = 1;
            exit(-1);
        }
        // 只是未分配物理页，则直接分配一个物理页
        char *mem;
        if((mem = kalloc()) == 0)
        {
            p->killed = 1;
            exit(-1);
        }
        memset(mem, 0, PGSIZE);
        uint64 pa = (uint64)mem; //提取kalloc出的mem的物理地址

        int perm = PTE_U;
        if (vma->prot & PROT_READ) perm |= PTE_R;
        if(vma->prot & PROT_WRITE) perm |= PTE_W;
        if(vma->prot & PROT_EXEC) perm |= PTE_X;
        //在用户页表上添加
```

```

        if(mappages(p->pagetable, va, PGSIZE, pa, per
m) != 0)
        {
            kfree(mem);
            exit(-1);
        }
        if(mappages(p->kpagetable, va, PGSIZE, pa, per
m & ~PTE_U) != 0)
        {
            vmunmap(p->pagetable, va, 1, 0);
            kfree(mem);
            exit(-1);
        }
    }
    ...
}

```

会首先查vma表，确认这一次缺页异常不是非法访问，而是由于vma未分配物理页导致的（注意，之前调用 `sys_mmap` 时，`mymmap` 已经从空闲vma链表中拿出一个交给该进程 `p->head` 下挂载了）；之后，会用 `mappages` 查询用户页表上，该引发异常的虚拟地址对应的 `pte` 是否可用，若 `v=0`，则将 `mem` 的物理地址添加权限后，直接修改该 `pte` 内容。

这样就完成了这次缺页异常对应的mmap页的物理绑定。

在刚开始，不涉及到超出内存中可用mmap页时，引发缺页异常都应该这样做，但在现实情况中，由于物理内存（DRAM，也就是分配的物理页所在的硬件中）大小有限，需要将多余的页换入磁盘中。这就是说，我们刚设计的全局交换区实际上在模拟磁盘在这里的作用。

因此需要修改 `usertrap` 的逻辑，使得：

1. 每次由于mmap引发的缺页异常，在需要分配物理页之前检查该进程当前在内存中的物理页数是否达到了上限。
2. 达到上限时，要进行换出页面的选择，执行换出操作。
3. 判断缺页地址从属哪一个页，这个页的status为0（未进入过内存）还是2（在交换区内，需要进入内存）

```

void usertrap(void)
{

```

```

...
else if (r_scause() == 13 || r_scause() == 15)
{
    uint64 va = r_stval();
    struct proc *p = myproc();
    va = PGROUNDDOWN(va);
    struct VMA *vma;
    for (vma = p->head.vm_next; vma != &p->head; v
ma = vma->vm_next)
        if (va >= vma->vm_start && va < vma->vm_end)
            break;
    if (vma == &p->head)
    {
        p->killed = 1;
        exit(-1);
    }
    struct VMA_page *page = addr2page(&p->head, v
a);
    if (page == 0)
    {
        p->killed = 1;
        exit(-1);
    }
    if (p->cur_page_in_mem >= p->max_page_in_mem)
    {
        struct VMA_page *victim = 0;
        victim = select_victim_page(&p->head);
        if (swap_out(p, victim) < 0)
        {
            p->killed = 1;
            exit(-1);
        }
    }

    if (page->status == 2)
    {
        // 页面在交换区内，需要换入到物理内存
        if (swap_in(p, page) < 0) // 这一步自动完成了cu

```


r_page_in_mem++的操作

```
{
    p->killed = 1;
    exit(-1);
}
}
else if (page->status == 0)
{
    // 页面尚未使用过, 执行全新分配
    char *mem;
    if ((mem = kalloc()) == 0)
    {
        p->killed = 1;
        exit(-1);
    }
    memset(mem, 0, PGSIZE);
    uint64 pa = (uint64)mem; // 提取kalloc出的mem
    的物理地址

    int perm = PTE_U;
    if (vma->prot & PROT_READ)
        perm |= PTE_R;
    if (vma->prot & PROT_WRITE)
        perm |= PTE_W;
    if (vma->prot & PROT_EXEC)
        perm |= PTE_X;
    // 在用户页表上添加
    if (mappages(p->pagetable, va, PGSIZE, pa, p
    erm) != 0)
    {
        kfree(mem);
        exit(-1);
    }
    if (mappages(p->kpagetable, va, PGSIZE, pa,
    perm & ~PTE_U) != 0)
    {
        vmunmap(p->pagetable, va, 1, 0);
        kfree(mem);
    }
}
```

```

        exit(-1);
    }
    p->cur_page_in_mem++;
    page->status = 1;           // 标记为在内存中
}
}
...
}

```

这里用到了一个 `addr2page`，在 `proc.c` 中定义的。

```

struct VMA_page *addr2page(struct VMA *head, uint64 addr)
{
    struct VMA *vma;
    for (vma = head->vm_next; vma != head; vma = vma->vm_next)
    {
        if (addr >= vma->vm_start && addr < vma->vm_end)
        {
            return &vma->pages[(addr - vma->vm_start) / PGSIZE];
        }
    }
    return 0;
}

```

然后这块的 `select_victim_page` 是一个简单的按照页面顺序挑选的占位内容，目的是检查现在所有的其他逻辑是否正确。

```

struct VMA_page *select_victim_page(struct VMA *head)
{
    struct VMA *v;
    struct VMA_page *victim = 0;

    // 遍历进程内部环形双向链表上挂载的所有 VMA 块

```

```

    for (v = head->vm_next; v != head; v = v->vm_next)
    {
        int npages = (v->vm_end - v->vm_start + PGSIZE - 1) / PGSIZE;

        // 遍历每一个 VMA 内部对应的物理页面信息
        for (int i = 0; i < npages; i++)
        {
            // 只有真正在内存中的页 (status == 1) 才能作为候选者
            if (v->pages[i].status == 1)
            {
                victim = &v->pages[i];
                printf("换出页面: %d\n", i);
                break;
            }
        }
    }

    if (victim == 0)
        panic("select_victim_page: no valid victim found");

    return victim;
};

```

▼ 现在这样实现以后，按照fifo测试样例的访问顺序要求，最终会有下面的结果，换出页面数为10次，和我们推测的顺序一样，说明其他逻辑都OK，接下来按照不同的测试点要求修改 `select_victim_page` 就行

访问页面	访问后内存状态	换出页面	是否换出
0	[0]	N	N
1	[0, 1]	N	N
2	[0, 1, 2]	N	N
3	[0, 1, 2, 3]	N	N
0	[0, 1, 2, 3]	N	N

1	[0, 1, 2, 3]	N	N
4	[1, 2, 3, 4]	0	Y
5	[2, 3, 4, 5]	1	Y
0	[0, 3, 4, 5]	2	Y
1	[1, 3, 4, 5]	0	Y
6	[3, 4, 5, 6]	1	Y
7	[4, 5, 6, 7]	3	Y
0	[0, 5, 6, 7]	4	Y
1	[1, 5, 6, 7]	0	Y
2	[2, 5, 6, 7]	1	Y
3	[3, 5, 6, 7]	2	Y

- `git push` 一下，然后在这个分支基础上去创建fifo或者LRU的branch。

▼ test_vm_fifo:

- 由于之前在 `proc.h` 中，为每一页已经实现了这个变量。

```
uint64 last_in_mem_time;      // 该页面进入内存的时间戳，用于FIFO算法
```

因此我们只需要在trap.c中，处理每一页的换入时加上这个变量的维护即可。而关于时间的定义，我们可以直接使用ticks。

```
void usertrap()
{
    ...
    p->cur_page_in_mem++;
    page->status = 1; // 标记为在内存中
}
acquire(&tickslock);
page->last_in_mem_time = ticks; // 更新进入内存时间戳
release(&tickslock);
...
}
```

随后，在select_victim_page中添加这个时间判断的逻辑就OK。

```
struct VMA_page *select_victim_page(struct VMA *head)
{
    struct VMA *v;
    struct VMA_page *victim = 0;
    int oldest_time = 0x7FFFFFFF; // 初始化为最大值
    int selected = -1;

    // 遍历进程内部环形双向链表上挂载的所有 VMA 块
    for (v = head->vm_next; v != head; v = v->vm_next)
    {
        int npages = (v->vm_end - v->vm_start + PGSIZE -
1) / PGSIZE;

        // 遍历每一个 VMA 内部对应的物理页面信息
        for (int i = 0; i < npages; i++)
        {
            // 只有真正在内存中的页 (status == 1) 才能作为候选者
            if (v->pages[i].status == 1)
            {
                if (v->pages[i].last_in_mem_time < oldest_time)
                {
                    oldest_time = v->pages[i].last_in_mem_time;
                    victim = &v->pages[i]; // 更新当前最老的页面指针
                    selected = i;
                    // printf("换出页面: %d\n", i);
                }
            }
        }
    }

    printf("换出页面: %d\n", selected);
    if (victim == 0)
        panic("select_victim_page: no valid victim found");
}
```

```
return victim;
};
```

▼ test_vm_lru:

- 首先需要实现一个系统调用 `#define SYS_lru_access_notify 54334`，这在测试样例中，是每次写入页面后调用的，这个是用来告诉内核，我最后一次访问的时间是这个时候。也就是说，我们可以用这个系统调用来维护。

```
uint64 last_access_time; // 该页面最后一次被访问的时间戳，
用于LRU算法
```

参考测试样例中的调用格式。

```
for (int i = 0; i < pattern_length; i++) {
    int page_index = access_pattern[i];
    mem[page_index * PAGE_SIZE] = 'A' + page_index;

    lru_access_notify((uint64)&mem[page_index * PAGE_SIZE]);
    // notify kernel the access action

    printf("Accessed page %d\n", page_index);
    sleep(1);
}
```

需要传递一个参数，这个参数指向了mmap某个页面的首地址。

```
uint64
sys_lru_access_notify()
{
    uint64 va;
    if (argaddr(0, &va) < 0)
    {
        return -1;
    }
    struct proc *p = myproc();
    struct VMA_page *page = addr2page(&p->head, va);
    if (page == 0)
    {
```

```

        return -1;
    }
    acquire(&tickslock);
    page->last_access_time = ticks; // 更新页面的最后访问时间戳
    release(&tickslock);
    return 0;
}

```

接下来，只需要对 `select_victim_page` 做修改就可以了。

```

struct VMA_page *select_victim_page(struct VMA *head)
{
    struct VMA *v;
    struct VMA_page *victim = 0;
    int oldest_time = 0x7fffffff; // 初始为一个很大的数，确保任何页面的时间戳都比它小

    // 遍历进程内部环形双向链表上挂载的所有 VMA 块
    for (v = head->vm_next; v != head; v = v->vm_next)
    {
        int npages = (v->vm_end - v->vm_start + PGSIZE - 1) / PGSIZE;

        // 遍历每一个 VMA 内部对应的物理页面信息
        for (int i = 0; i < npages; i++)
        {
            // 只有真正在内存中的页 (status == 1) 才能作为候选者
            if (v->pages[i].status == 1)
            {
                if (v->pages[i].last_access_time < oldest_time)
                {
                    oldest_time = v->pages[i].last_access_time;
                    victim = &v->pages[i];
                }
            }
        }
    }
}

```

```
}  
if (victim == 0)  
    panic("select_victim_page: no valid victim found");  
  
return victim;  
};
```